Learning Objectives

This lab lets you practice writing programs using dictionaries.

Estimated Size

1 program file: `dictionaries.py`, 2 functions, and less than 20 lines of code in each function.

Setup

In your Python files, you must include author information. On the first line, add a `# Author` comment line, where you mention your full name. Similarly, include your email and Spire ID on the `# Email` and `# Spire ID` comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author   : John Snow
# Email    : jsnow@umass.edu
# Spire ID : 12345678
```

1. Implement function `most_frequent_element` (8 points)

In many applications, we need to find which item appears the most in a list — for example, finding the most common word in text or the most frequent number in survey data.

Write a function, called `most_frequent_element`, which should:

- Take a **list of elements** (numbers, strings, or both) as a parameter.
- Return the **element** that appears most frequently in the list.
- If there are multiple elements with the same highest frequency, return **any one** of them.

Hints

- You can use a **dictionary** to count how many times each element appears.
- Loop through the list and update the counts.
- Then, find the key with the **largest count**.
- Handle the case when the list is empty by returning `None`.

Below are examples of running a correct implementation of this function:

```
print(most_frequent_element([3, 1, 3, 2, 3, 1, 2, 2, 2]))
# 2

print(most_frequent_element(["apple", "banana", "apple", "orange", "banana", "apple"]))
# 'apple'

print(most_frequent_element([1, 2, 3, 4]))
```

```
# 1, 2, 3, or 4 (any valid answer)

print(most_frequent_element([]))
# None
```

2. Implement function `greet_user` (7 points)

Write a function that greets a user based on their stored information in a nested dictionary.

The function takes:

- `users`: a nested dictionary where:
  - keys are user IDs (strings)
  - values are dictionaries with possible keys `"name"`, `"language"`, and `"age"`
- `user_id`: the ID of the user to greet
- `default_lang`: a string (default `"en"`) that is used if the user's language is missing
- `default_age`: an integer (default `18`) that is used if the user's age is missing

The function should:

1. Return a greeting in the user's preferred language (or the default language if missing).
2. Use a **child greeting** if the user's age is below 18.
3. Return `"User not found."` if the ID is not in the dictionary.

Language	Adult greeting	Child greeting
`"en"`	`"Hello, {name}!"`	`"Hey there, {name}!"`
`"es"`	`"Hola, {name}!"`	`"¡Hola, pequeño/a {name}!"`
`"fr"`	`"Bonjour, {name}!"`	`"Salut, {name}!"`

Below are examples of running a correct implementation of this function:

```
users = {
    "u1": {"name": "Alice", "language": "en", "age": 25},
    "u2": {"name": "Carlos", "language": "es", "age": 12},
    "u3": {"name": "Marie", "language": "fr"},
    "u4": {"name": "UnknownUser"}  # Missing info
}
print(greet_user(users, "u1"))
# Hello, Alice!

print(greet_user(users, "u2"))
# ¡Hola, pequeño/a Carlos!

print(greet_user(users, "u3"))
```

```
# Bonjour, Marie!   (default_age=18 → adult greeting)

print(greet_user(users, "u4", default_lang="es"))
# Hola, UnknownUser!   (uses default language)

print(greet_user(users, "u5"))
# User not found.
```

4. Submission to Gradescope

Log in to Gradescope, select **Extra Credit Practice Lab 08**, and submit the required file. Wait until the autograder finishes running and returns the result to you. You can resubmit as many times as you like before the deadline.

(Common Paragraph) Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems, and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.

You may not use any code that is posted on the internet. If you are not sure it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the course as well as online resources. It is very easy for us to detect similar submissions and will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the UMass Academic Honesty Policy and Procedures so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty you should re-read the course policies. We assume you have read the course policies in detail and by submitting this project you have provided your virtual signature in agreement with these policies.